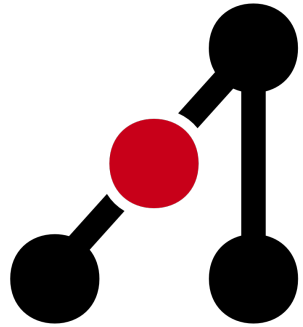




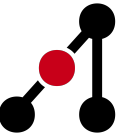
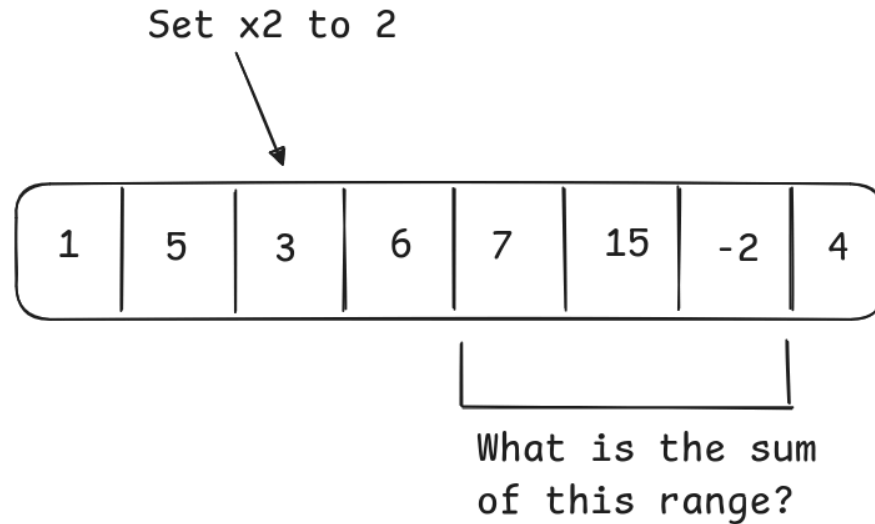
**Adelaide Competitive
Programming Club**

Round 2 Workshop A

All About Segment Trees

Motivating the Problem: Simple Range Queries

We have n integers. Over time we want to modify individual elements, and ask what the sum of any range of elements is. Both operations should be efficient as possible.



Motivating the Problem: Simple Range Queries

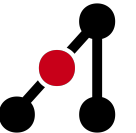
Dumb solution. Store the array as an array.

Update: Just update the element in an array - $O(1)$

Query: Just loop over l-r and sum - potentially $O(n)$

Clearly $O(n)$ query isn't very good, but at least updates are fast. Maybe if you have up to 10 queries and up to 10^5 updates or something this is a good approach...

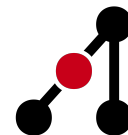
What if we want faster queries?



Motivating the Problem: Simple Range Queries

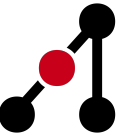
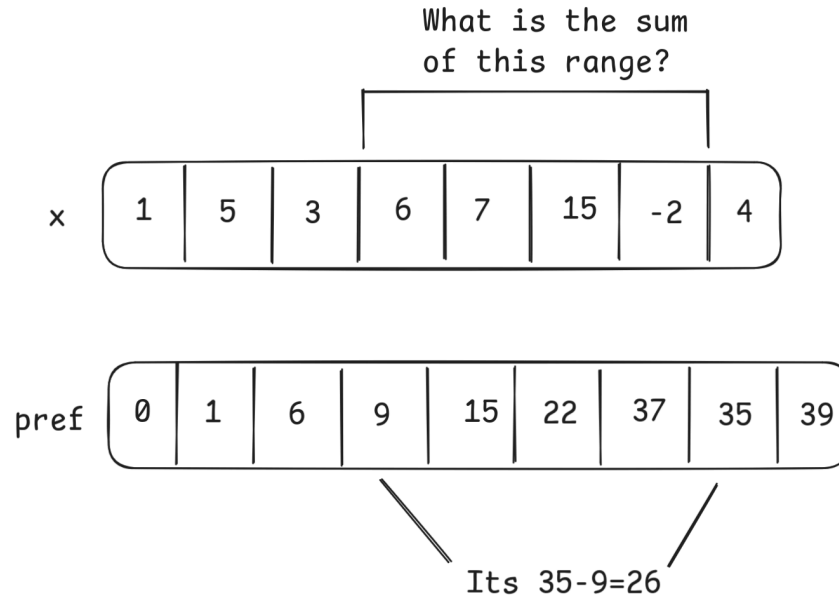
Smarter solution. Store prefix sums. Make a new array. Each index i in this array is the sum of all elements **before** index i in the original array. Notice that the length is $n + 1$, as the first element is necessarily 0 (the sum of all elements before the first element is the sum of no elements).

x	1	5	3	6	7	15	-2	4	
pref	0	1	6	9	15	22	37	35	39



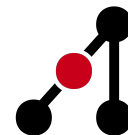
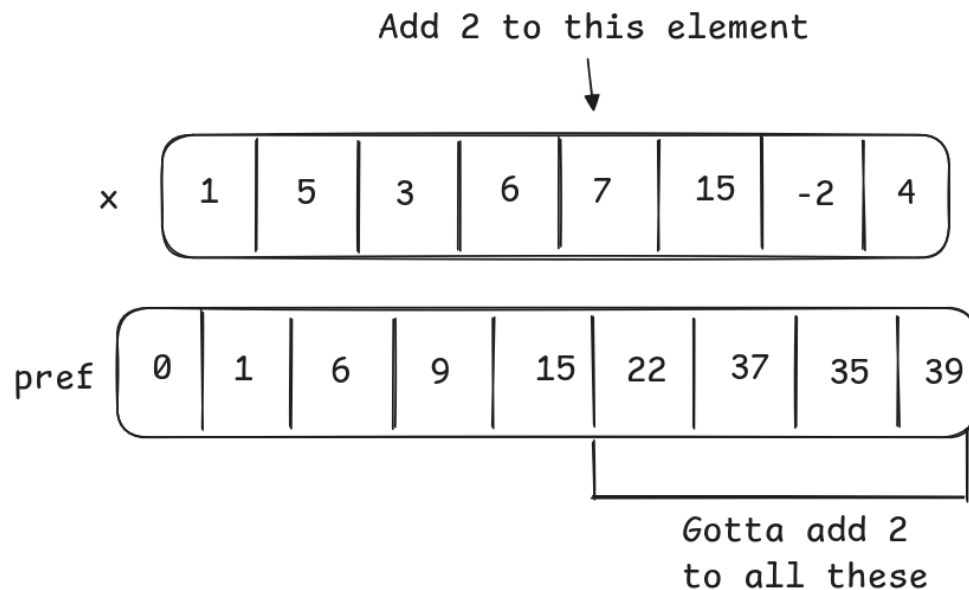
Motivating the Problem: Simple Range Queries

We can perform range queries by simply subtracting the bounds in $O(1)$ time. Remember to add 1 to the right bound for an inclusive sum.



Motivating the Problem: Simple Range Queries

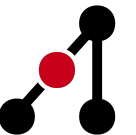
However, to do a point update, we need to update every single element after it - $O(n)$ worst case.



Introducing: Segment Tree

We noticed: raw array has $O(1)$ updates, $O(n)$ queries. Prefix array has $O(n)$ updates, $O(1)$ queries. What if we could do both in $O(\log(n))$?

- A segment tree is a binary tree where each node represents some subarray of the array.
- The root is the entire array $[0, n)$.
- Leaves are single array elements $[i, i]$.
- The children of a node are the subarray split in two. I.e. if the root is $[0, 5]$, the children are something like $[0, 2]$ and $[3, 5]$.
- It requires linear amount of vertices, usually $4n$ allocated, so $O(n)$ memory.

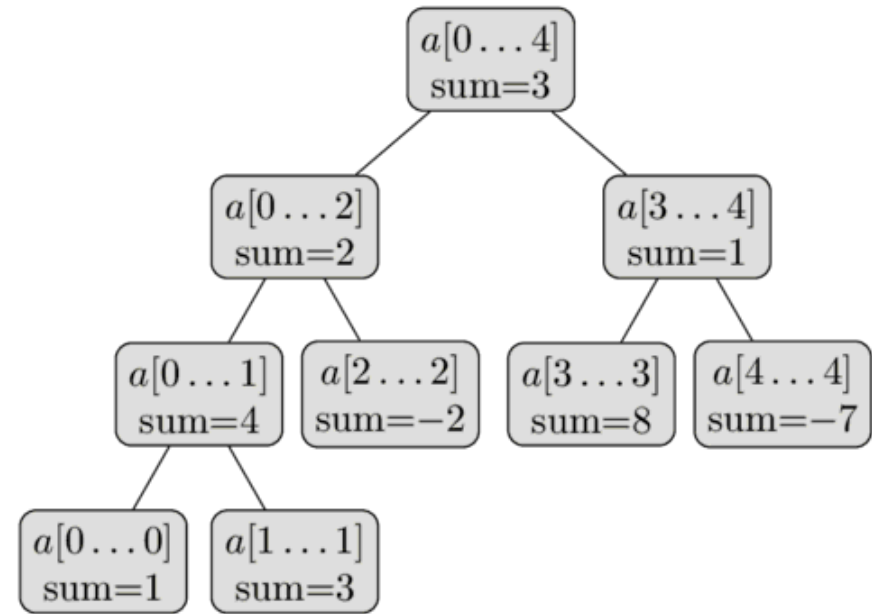


Introducing: Segment Tree

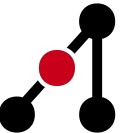
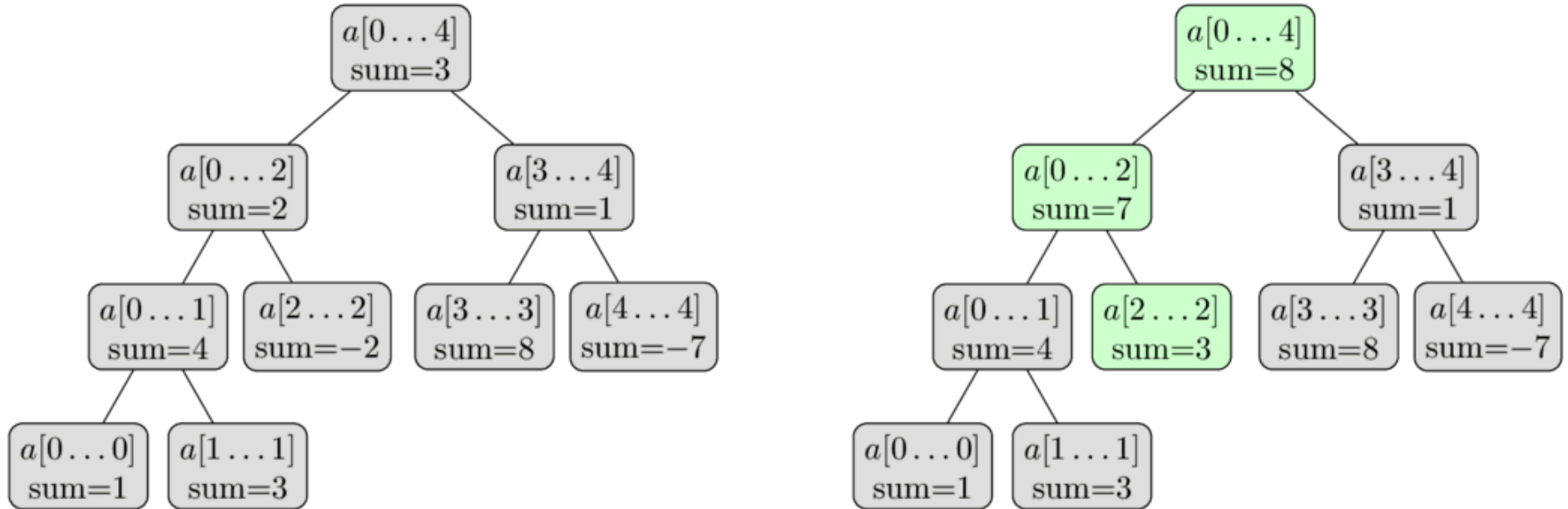
You can see the segment tree representation for $[1, 3, -2, 8, -7]$. Since n is not a power of two, the binary tree is not complete.

Using this tree, it is trivial to see how we would do a point update. Simply walk from the root down towards the leaf that represents the element, and modify every segment visited. There are $O(\log(n))$ in the path.

Think about how you might do the query...



Say we want to increase $a[2]$ by 5, creating $[1, 3, 3, 8, 7]$. Left is the original tree, right is the updated one, with the required path in green. You can imagine this recursively: if node is $[x, x]$, update and return. Otherwise, update and recurse on half that contains x . I hope you can see why this is $O(\log(n))$



Now to perform a query. For any l, r , we want to find sum of $a[l..r]$ inclusive on $O(\log(n))$. We will use our precomputed segments, and imagine it recursively, starting at the root.

Imagine we are currently at the node containing segment $[tl..tr]$ and we are looking for $[l..r]$. There are a few cases.

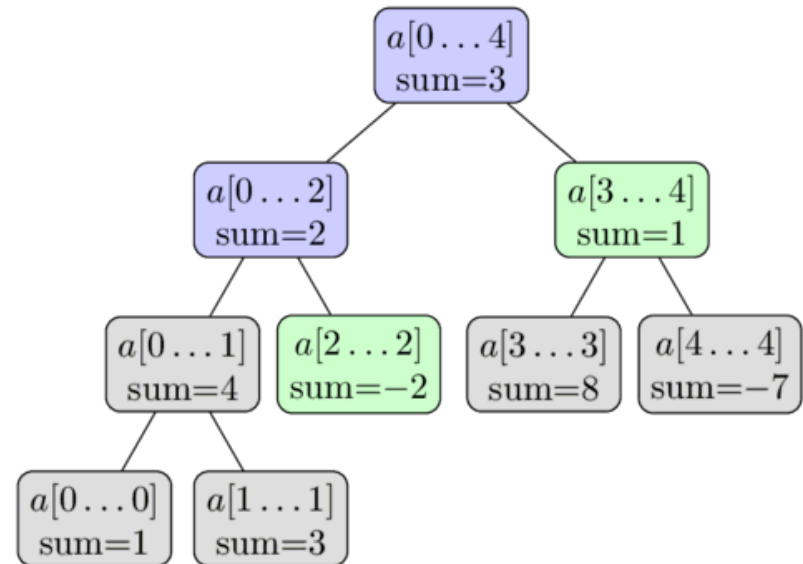
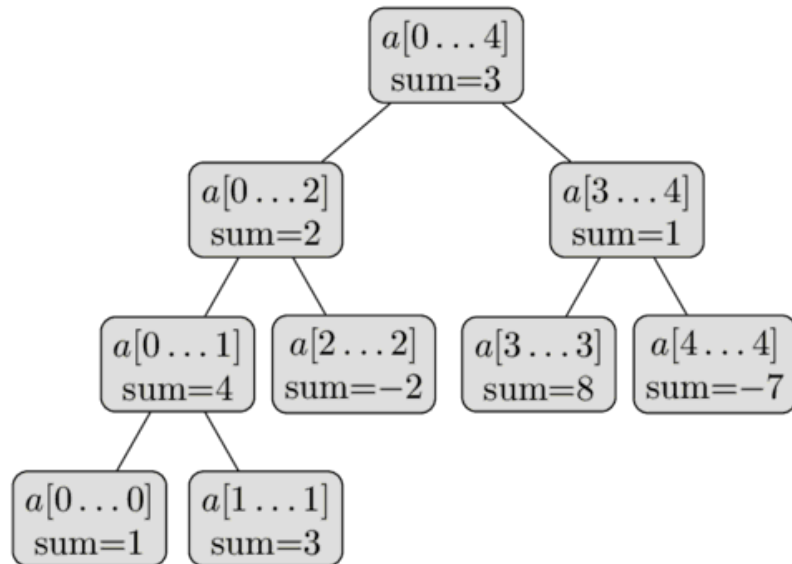
- First, the intervals are the same $tl = l$ and $tr = r$. Just return stored values.
- Maybe $[l..r]$ lies fully in one of the halves of $[tl..tr]$.
- Finally, the interval might overlap the midpoint
 - In this case, we have to do two recursive calls and merge results by summing
 - One for $[l..tm]$ on the left child
 - One for $[tm + 1..r]$ on the right child

These 3 cases are all that is possible, so they allow us to do the query.



Here we are querying from index 2 to 4. Reminder the array is $[1, -3, -2, 8, 7]$

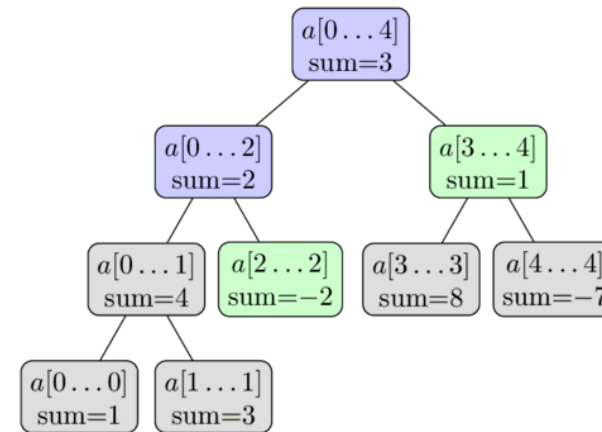
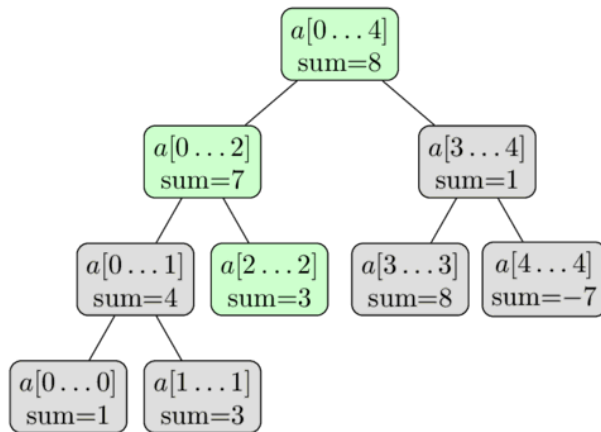
The root is case 3, recurse for $[2..2]$ on left node and $[3..4]$ on right node. The left node is the second case, $[2..2]$ is fully in right child so just call that. Now the remaining two are the green leaves, which perfectly overlap, so they are returned. The result is $-2 + 1 = -1$.



Time Complexity

It is clear to see why the update is $O(\log(n))$: the levels of the tree partition the array, so each point is only in one node per level. The height is $\log(n)$.

For queries, the splitting is suspicious. It might seem like we could have to recurse more than logarithmic time. However, if you analyse the number of calls per level, there can't be more than 4 as then the intervals would overlap. 4 times the height is still bounded by $\log(n)$



Constructing SegTrees

These operations only work once we already have the segtree setup. How do we create the tree given some array?

- First, we don't usually use pointers for the tree, we can instead allocate $4n$ vector, say 1 is the root, and say for index i the children are $2i$ and $2i + 1$. This gives better cache locality.
- You can think of the segtree as being inductively defined.
 - The base case is how the leaves are populated. In our case, it is the corresponding array value.
 - The recursive case is how the nodes are merged together going up. In our case, it is just adding the values.

Thus, starting from the root, we can call recursively. If the interval is 1 big, i.e. a leaf, we should just return the value. If it isn't, we should divide the interval and recurse twice on each child, then merge them and assign the value.

This will construct our segtree in $O(n)$ time, as the number of total nodes is bounded by $4n$.



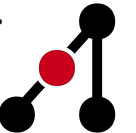
Worked Example of Construction $[1, 2, 3, 4, 5]$

We are building $[1, 2, 3, 4, 5]$. We show the tree array, and the current DFS call.



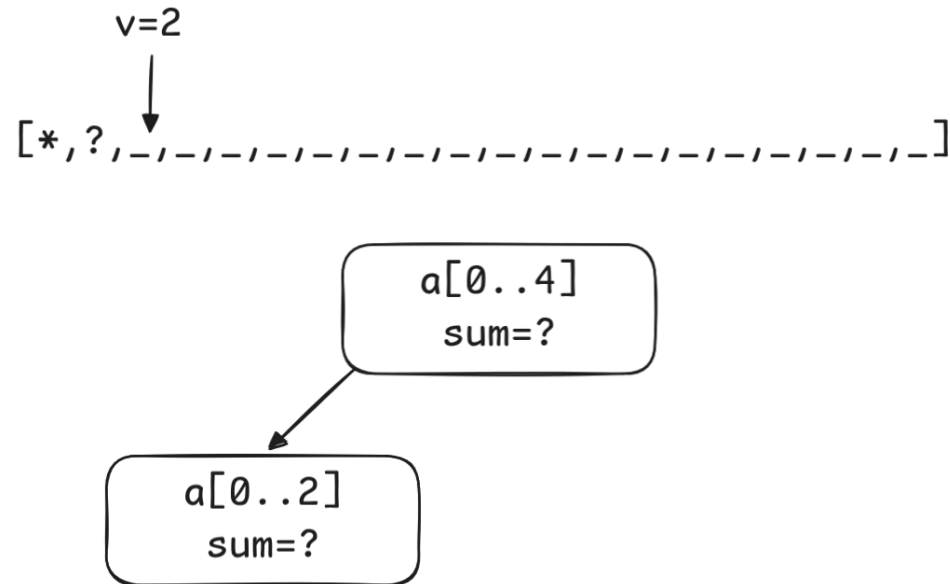
a[0..4]
sum=?

Clearly, $tl \neq tr$, so we calculate $tm = \frac{0+5}{2} = 2$ and recurse for children. First, we go to left child $[0, ..2]$.

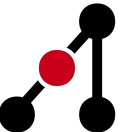


Worked Example of Construction $[1, 2, 3, 4, 5]$

Now, $v = 2$. This is because recall for any i the left child is index $2i$. Corresponds to $[0..2]$.



Again $tl \neq tr$. So we recurse on children $v = 4, [0..1]$ and $v = 5, [2..2]$.

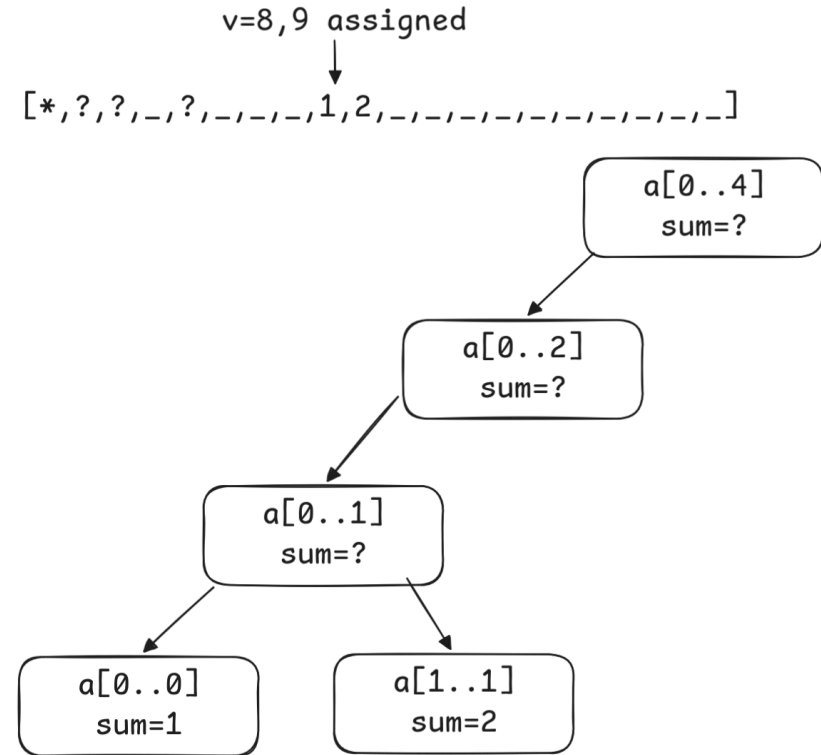


Worked Example of Construction [1, 2, 3, 4, 5]

We are at $v = 4$. Here we will skip some steps.

Again, the array will be divided into $v = 8, [0..0]$ and $v = 9, [1..1]$.

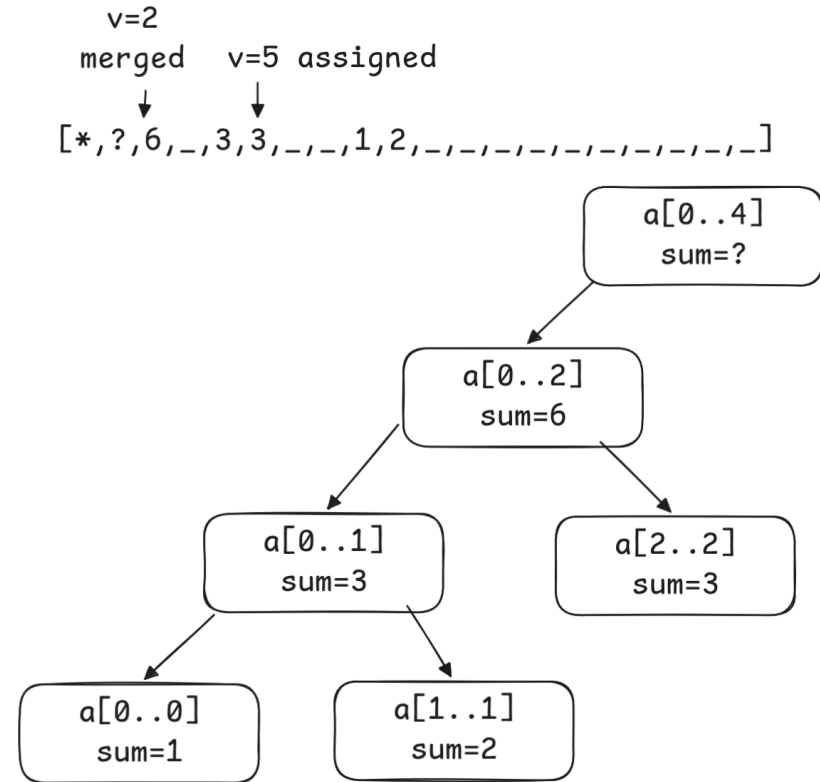
For these ones though, $tl = tr$, so the sums will be populated. Now, we know the values of both children, so we need to merge



Worked Example of Construction [1, 2, 3, 4, 5]

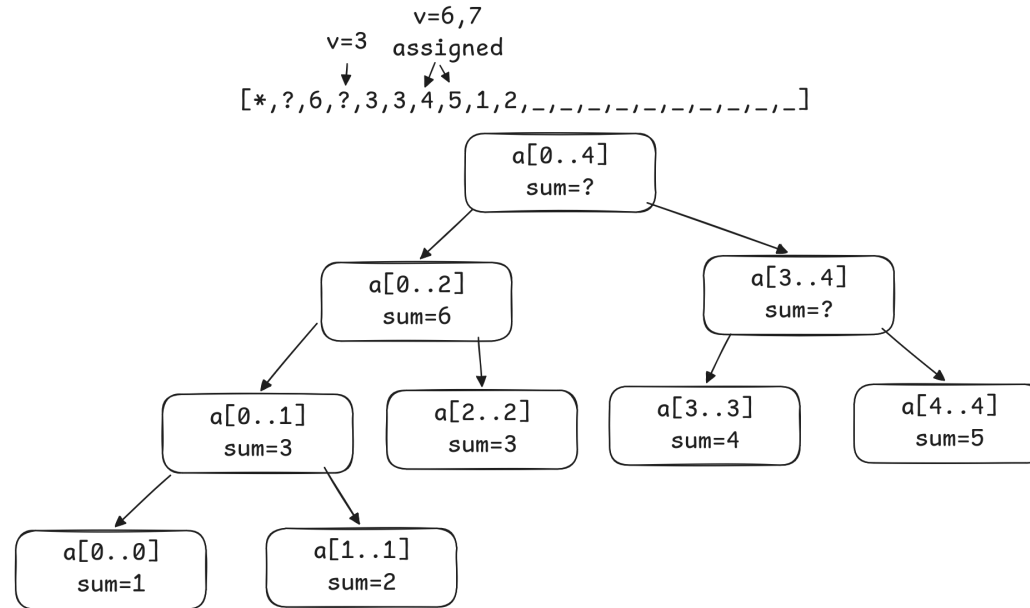
Now, since both children have finished, the call at $v = 4$ will conclude, assigning $\text{sum} = 1 + 2 = 3$.

Then, the DFS will go back to $v = 2$, which will call its right child, $v = 3$, [2..2]. This is also a leaf, so it will be filled, concluding $v = 2$'s call after merging $3 + 3 = 6$.



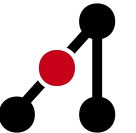
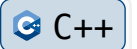
Worked Example of Construction $[1, 2, 3, 4, 5]$

Finally, all of $v = 3, [3..4]$'s children are assigned. Then all of $v = 1, [0..4]$ children are done. After merging, the whole segtree is complete.



Code Snippet: Construction

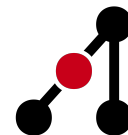
```
int n, t[4*MAXN];  
void build(int a[], int v, int tl, int tr) {  
    if (tl == tr) {  
        t[v] = a[tl]; // Base case  
    } else {  
        int tm = (tl + tr) / 2;  
        build(a, v*2, tl, tm);  
        build(a, v*2+1, tm+1, tr);  
        t[v] = t[v*2] + t[v*2+1]; // Merge children  
    }  
}
```



Code Snippet: Sum

In order to simplify things, we just always make two recursive calls. If the entire $[l, r]$ is within one child, the other recursive call will just instantly return.

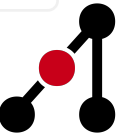
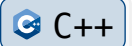
```
int sum(int v, int tl, int tr, int l, int r) {  
    if (l > r)  
        return 0;  
    if (l == tl && r == tr) {  
        return t[v];  
    }  
    int tm = (tl + tr) / 2;  
    return sum(v*2, tl, tm, l, min(r, tm))  
        + sum(v*2+1, tm+1, tr, max(l, tm+1), r);  
}
```



Code Snippet: Update

Here we only ever need a single recursive call, as one element can't overlap the midpoint.

```
void update(int v, int tl, int tr, int pos, int new_val) {  
    if (tl == tr) {  
        t[v] = new_val;  
    } else {  
        int tm = (tl + tr) / 2;  
        if (pos <= tm)  
            update(v*2, tl, tm, pos, new_val);  
        else  
            update(v*2+1, tm+1, tr, pos, new_val);  
        t[v] = t[v*2] + t[v*2+1];  
    }  
}
```



Using Segment Trees

Now you might have some idea about a sum segment tree, but many problems won't just ask you for range sum and point update. Segment trees are very flexible, and can be adapted to solve many problems.

- First and foremost, you can use different merge functions! Your segtree could compute range XORs, range min and max, range GCD or LCM, and many more. Using these just equates to swapping the `+` out in the above functions. You can also store other data types besides just integers.
- You could make a 2-dimensional segtree, allowing rectangular queries of a 2d coordinate space.
- You could make more memory-efficient versions. Before you could've noticed not all $4n$ slots were used.



Example Time: A Game of Clones

<https://aucpl.com/problem/agameofclones>. Think about how you might solve this. Don't worry about the big integers or actually implementing it, just think about it and discuss.

As the ruler of the Seven Kingdoms and protector of the realm, Ritisha has noticed a recurring pattern in the wars between the houses. These wars follow a unique set of rules that determine their outcome.

There are n houses, numbered from 1 to n , each with a certain initial population S_i . When a war breaks out, the houses simultaneously battle in pairs:

- House 1 battles house 2.
- House 3 battles house 4.
- And so on.

The winners of each pair then move on to battle against each other in a tournament-style format, until only one house remains as the champion. The champion's population doubles, while all other houses retain their current population. Then, the next war begins.

The outcome of each battle depends on the season:

- In summer, the house with the larger population wins.
- In winter, the house with the smaller population wins.
- If their populations are the same, the leftmost house wins always.

Each set of battles lasts two seasons. If one set of battles occurs in summer, the next one will be in winter, and so on.

As protector of the realm, Ritisha wants to predict the champion of each war. Can you help her figure this out?

Input

The first line of each test case consists of two integers n ($1 \leq n \leq 131072$), the number of houses in the realm, and m ($1 \leq m \leq 10^5$), the number of wars that break out. n is always a power of 2 ($n = 2^k$, where k is an integer).

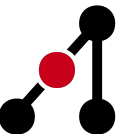
The next line contains n integers, $S_1 \dots S_n$ ($1 \leq S_i \leq 100$), where S_i indicates the initial population of the i th house.

Output

For each test case, output the winner of each war.

Clarifications

- The first war begins in summer.



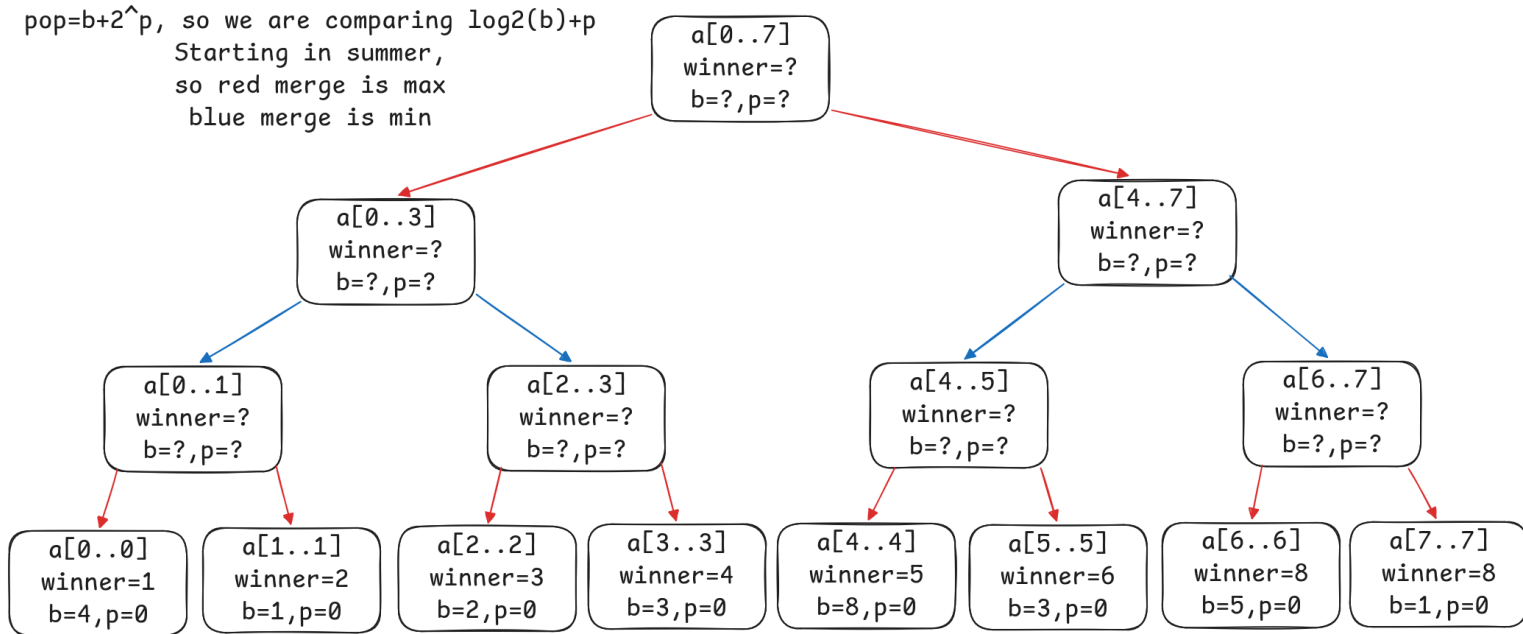
Solving A Game of Clones

- Populations may double up to 10^5 , just store base and power of two, compare using $\log(b) + p$.
- Physically model tournament structure as segment tree, computed value of node is winner of segment.
- In the leaves obviously the winner is just the point. To merge upwards, we need to find winner of battle.
- This depends on additional context: the height. If the height of the node is odd/even, the merge is min, if it is even/odd it is max.
- The parity of this depends on if the tournament starts in summer or winter. To model this, we can use two separate segment trees, one for starting in summer and one for starting in winter. Note if the height is odd we only need one segtree really.
- After each round we can do a point update on both trees to increment the p value of the winner.



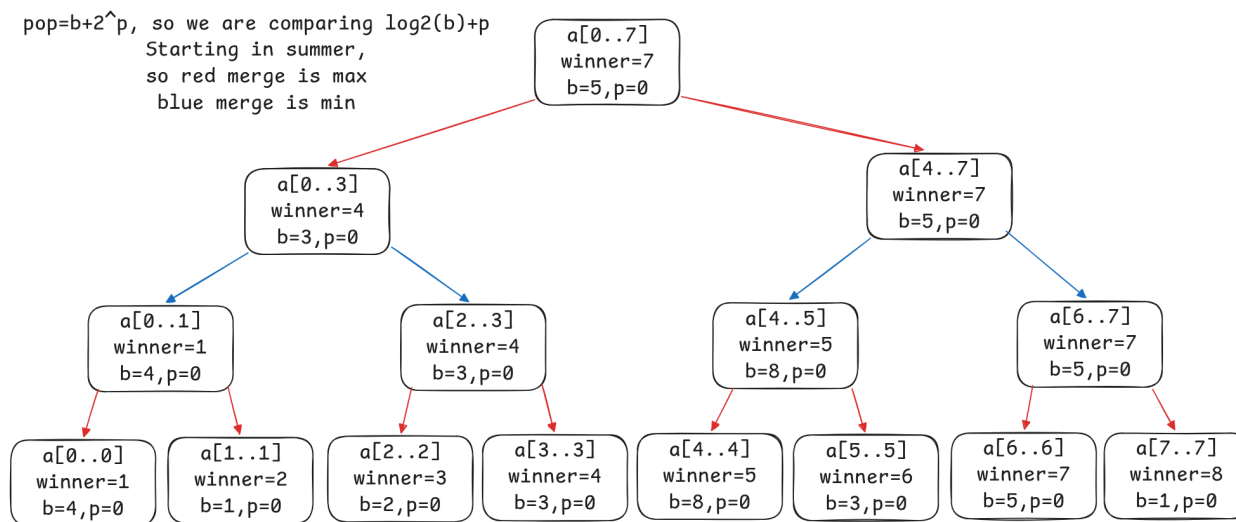
Solving A Game of Clones

Example: 8 houses, 2 rounds, starting pops 4 1 2 3 8 3 5 1. Note $pop = b \times 2^p$ not plus as shown.

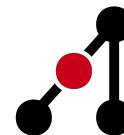


Solving A Game of Clones

We fill in the computed values recursively and see that round 1 winner is house 7!.

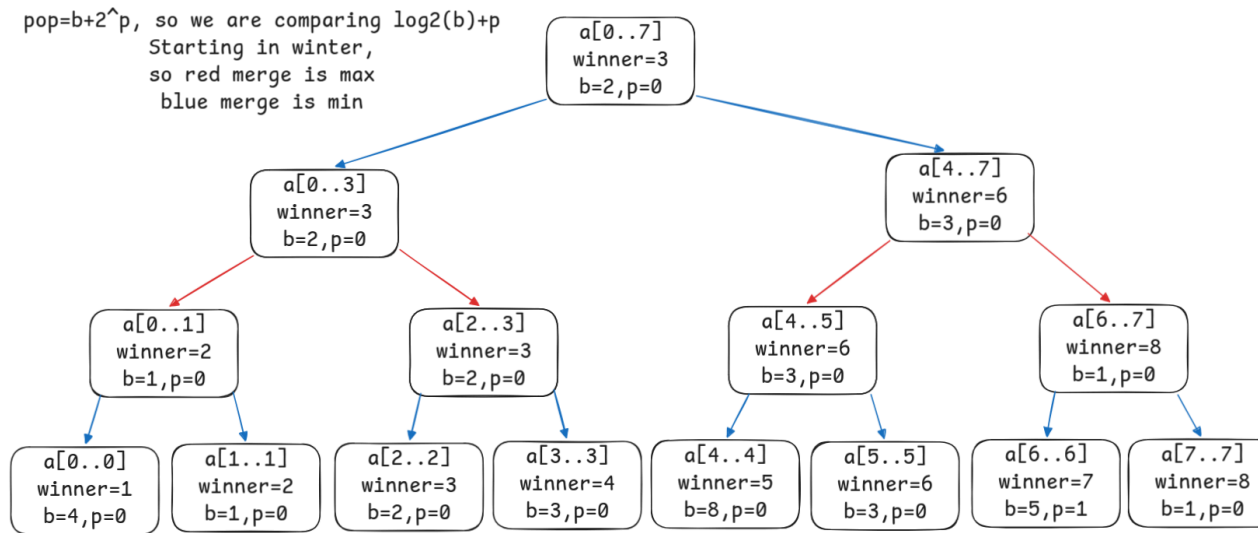


Now, we should do a point update on $a[6]$ to increment p . Also notice there were an odd number of levels, so we need the second segtree for the next battle. I.e. bottom level should be min not max.

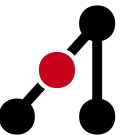


Solving A Game of Clones

This is the filled out starting-in-winter version of our tree, incorporating the point update for 7.



We correctly computed the outputs for this case: 7 then 3. Read editorial [here](#).



Another Example from AtCoder

ABC 424 problem F - adding chords: https://atcoder.jp/contests/abc424/tasks/abc424_f

There are N points equally spaced around a circle, numbered $1, 2, \dots, N$ clockwise.

You are given Q queries of the following form; process them in order.

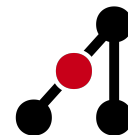
- Draw the line segment connecting points A_i and B_i . However, if this segment intersects a segment that has already been drawn, do not draw it.

Here, it is guaranteed that the $2Q$ integers $A_1, \dots, A_Q, B_1, \dots, B_Q$ are all distinct.

For each query, answer whether the segment was drawn.

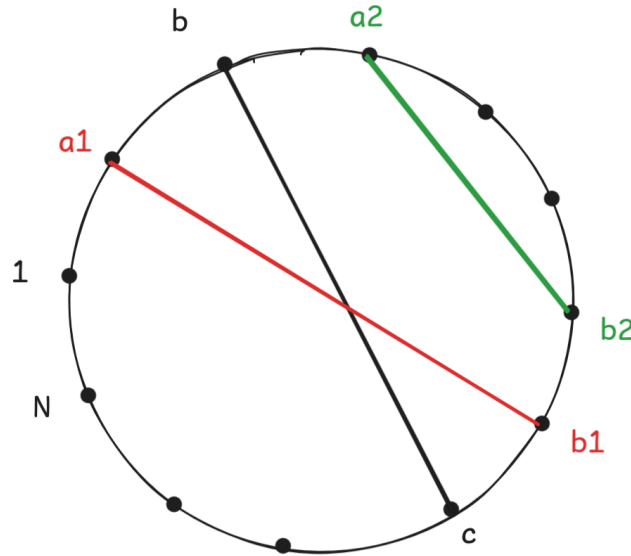
The input is just N, Q and A_i, B_i for $i \in [1, Q]$.

There are a number of ways to solve this with segtrees - I will present the one I think the most elegant in a bit.



Another Example from AtCoder

Here is an observation: there exists a chord from points $a - b$. Say we are drawing the chord $c - d$ and checking if it intersects. It only intersects if exactly one of a, b is in the range $[c, d]$. If both are in the range, or both are not in the range, it will not intersect.



Another Example from AtCoder

Using this observation, we will solve the problem using an XOR segtree over $1 - N$.

- Each value will begin as 0.
- When we get a query, we will check if the XOR sum between A_i and B_i is 0. If it is zero, there is no intersecting chords, and we can draw it. If it is nonzero, there is an intersection and we cannot draw it.
- To 'draw' the chord, we need some arbitrary random number. Then, we will do a point update to set both A_i and B_i to that number.

This leverages the fact that XORing the same number twice will create 0, so if the overall range sum of an interval is 0, there must either be no endpoints of a given chord, or both endpoints.

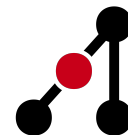
If you are using sufficiently large random numbers, the chance of a collision is low enough to pass. TC is $O(Q \log(n))$. <https://atcoder.jp/contests/abc424/editorial/13940>



More SegTrees

- Lazy propagation segtree! Previously, we can only do point updates. With this modification, we can modify whole ranges in $O(\log(n))$. We basically store for each node how many times we should add to the whole segment, and then 'lazily propagate' that change downwards during later queries.

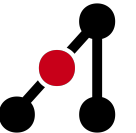
https://cp-algorithms.com/data_structures/segment_tree.html#range-updates-lazy-propagation



More SegTrees

- Fenwick Trees! Very similar to segment trees, it can do range queries and point updates. It is a bit simpler to implement, but it is based in some tricky bit manip so harder to understand. It is more memory efficient and probably a bit faster.

https://cp-algorithms.com/data_structures/fenwick.html



More SegTrees

- Persistent Segment Tree! It is possible to store the history of values at each node, so we can perform queries at any point in time, by maintaining a 'timestamp' value. We do this by dynamically allocating the nodes, and creating new root node for each query.

https://cp-algorithms.com/data_structures/segment_tree.html#preserving-the-history-of-its-values-persistent-segment-tree



More SegTrees

- As mentioned, segment or fenwick trees can become 2-dimensional. This allows rectangular queries, i.e. each point in x -space of the outer segtree corresponds to a whole inner y -segtree. For instance, in a grid, you can ask for the sum of rectangular regions.

https://cp-algorithms.com/data_structures/segment_tree.html#generalization-to-higher-dimensions

- You can also make regular segment trees more memory efficient, as $4n$ is quite a lot. We could dynamically allocate all nodes instead of using a flat array. Also, for instance, if we have $|x| \leq 10^9$ but only 10^5 we can do 'coordinate compression' by assigning each unique value an index.

https://cp-algorithms.com/data_structures/segment_tree.html#memory-efficient-implementation



Feedback Form

We want to get better at giving engaging and educational workshops. Please leave us feedback so we can know how we're doing and what to improve!

Check `#aucpl-announcements` on Discord for the form, but feel free to talk to us in person or on Discord generally.



Next Up!

Next workshop is scheduled for Thursday, May 7th.

The topic is **All About Computational Geometry!**

